Report: Optimising NYC Taxi Operations

Include your visualisations, analysis, results, insights, and outcomes. Explain your methodology and approach to the tasks. Add your conclusions to the sections.

1. Data Preparation

1.1. Loading the dataset

1.1.1. Sample the data and combine the files

To create a representative yearly dataset while keeping computation manageable, the data was sampled using a **time-aware strategy**:

- For each monthly file:
  - Extract `pickup_date` and `pickup_hour` from `tpep_pickup_datetime`
  - For each **date** and each of the **24 hours**, randomly sample a small fraction (5%) of trips for that hour
  - Append these samples to build the sampled month, then append to the yearly dataframe

This approach ensures the sampled dataset preserves:

- **hourly pickup patterns** (e.g., commute peaks vs late-night lows)
- **day-to-day variation** within a month
  Instead of sampling only “overall rows”, sampling by **date + hour** prevents over-representing high-traffic hours/days and under-representing low-traffic periods.

After sampling all months, the dataset was **capped to ~300,000 rows** (target range 250k–300k) to match the assignment guideline and ensure smooth downstream analysis.

```
Done sampling: 2023-9.parquet | sampled rows so far = 140875
Done sampling: 2023-5.parquet | sampled rows so far = 285333
Done sampling: 2023-6.parquet | sampled rows so far = 448243
Done sampling: 2023-2.parquet | sampled rows so far = 616939
Done sampling: 2023-1.parquet | sampled rows so far = 769026
Done sampling: 2023-8.parquet | sampled rows so far = 912808
Done sampling: 2023-7.parquet | sampled rows so far = 1086876
Done sampling: 2023-10.parquet | sampled rows so far = 1261131
Done sampling: 2023-12.parquet | sampled rows so far = 1427840
Done sampling: 2023-3.parquet | sampled rows so far = 1591626
Done sampling: 2023-4.parquet | sampled rows so far = 1731267
Done sampling: 2023-11.parquet | sampled rows so far = 1896400
```

Storing the sampled dataset

Finally:

- Helper columns (`pickup_date`, `pickup_hour`) used for sampling were dropped.
- The sampled dataset was saved as a **Parquet file** with **Snappy compression**, which is efficient for storage and faster to reload compared to CSV.

Outcome of Section 1: A single sampled yearly dataset of **300,000 rows** was created and saved (`nyc_taxi_sampled_300k.parquet`) for all further EDA and analysis.

2. Data Cleaning

2.1. Fixing Columns

2.1.1. Fix the index

After sampling and saving, the dataframe index was reset to ensure it is clean and continuous. This avoids downstream issues in filtering, joining, and plotting (especially after row drops and concatenations).

Action taken: Reset index using `reset_index(drop=True)`.

2.1.2. Combine the two airport_fee columns

The dataset contained **two airport fee columns**: `Airport_fee` and `airport_fee`. This appears to be a naming inconsistency (case difference) rather than separate variables. To fix this, the two columns were consolidated into a single column:

- A unified `airport_fee` column was created using the non-null values from `Airport_fee` first and falling back to `airport_fee` where needed.
- Missing values were filled with `0` after combination (treating "missing fee" as "fee not applied").
- The duplicate `Airport_fee` column was then dropped.

Outcome: A single consistent airport fee field (`airport_fee`) is available for all records, eliminating ambiguity and simplifying analysis.

2.1.3. Fix columns with negative (monetary) values

Next, I checked for negative values across monetary fields such as:
`fare_amount, extra, mta_tax, tip_amount, tolls_amount, improvement_surcharge, total_amount, congestion_surcharge, airport_fee`.

The analysis showed:

- `fare_amount` did **not** have negative values in the sampled dataset.
- Negative values were present in other monetary fields (notably `total_amount`, `improvement_surcharge`, `mta_tax`, and `congestion_surcharge`), which suggests these entries are likely **refunds/adjustments or incorrectly recorded transactions**.

	fare_amount	extra	mta_tax	tip_amount	tolls_amount	improvement_surcharge	total_amount	congestion_surcharge	airport_fee
14296	0.0	0.0	-0.5	0.0	0.0	-1.0	-5.25	-2.5	-1.25
48075	0.0	0.0	-0.5	0.0	0.0	-1.0	-3.25	0.0	-1.75
66010	0.0	0.0	-0.5	0.0	0.0	-1.0	-1.50	0.0	0.00
70515	0.0	0.0	-0.5	0.0	0.0	-1.0	-2.75	0.0	-1.25
72012	0.0	0.0	-0.5	0.0	0.0	-1.0	-4.00	-2.5	0.00
128079	0.0	0.0	-0.5	0.0	0.0	-1.0	-4.00	-2.5	0.00
139063	0.0	0.0	-0.5	0.0	0.0	-1.0	-4.00	-2.5	0.00
184046	0.0	0.0	-0.5	0.0	0.0	-1.0	-4.00	-2.5	0.00
240442	0.0	0.0	-0.5	0.0	0.0	-1.0	-4.00	-2.5	0.00

To ensure consistency for standard revenue and fare analysis, all rows where **any monetary field was negative** were treated as invalid for this EDA and removed.

Outcome: After removing these anomalous monetary rows, the dataset size reduced slightly (final shape shown as **~299,991 rows**), and the remaining dataset is suitable for revenue, pricing, and surcharge analysis without distortions from negative adjustments.

2.2. Handling Missing Values

2.2.1. Find the proportion of missing values in each column

The first step was to compute the proportion of missing values per column. The dataset showed missingness concentrated in a small set of features:

- `passenger_count`
- `RatecodeID`
- `store_and_fwd_flag`
- `congestion_surcharge`

Each of these had approximately **3.4% missing values** (0.034321 proportion). This indicates the missingness is relatively small, but still important to treat because these fields directly affect demand segmentation (passenger_count), fare rules (RatecodeID), operations (store_and_fwd_flag), and pricing (congestion_surcharge).

2.2.2. Handling missing values in passenger_count

For `passenger_count`, missing values were first inspected, and then imputed using the **mode** (most frequent value). Since taxi trips most commonly carry **1 passenger**, mode imputation is reasonable and avoids biasing averages upward.

Additionally, records where `passenger_count == 0` were also treated as invalid/improper entries for a passenger trip and replaced with the same mode value. This ensures the passenger count field remains meaningful for downstream analysis.

Outcome: `passenger_count` contained no remaining NaNs, and zero counts were corrected.

2.2.3. Handle missing values in RatecodeID

`RatecodeID` is a categorical-style numeric variable representing fare rule categories (standard, JFK, Newark, etc.). Missing values were imputed using the **mode**, which effectively assigns the most common fare rule to missing cases (typically the standard rate category).

Outcome: `RatecodeID` became complete with no missing values

2.2.4. Impute NaN in congestion_surcharge

`congestion_surcharge` was also imputed using the mode. Since this surcharge is often either a fixed amount (e.g., 2.5) or 0 depending on time/location/payment rules, mode imputation is a practical approach that keeps values realistic rather than introducing arbitrary averages.

Outcome: `congestion_surcharge` was filled and made complete.

Handling remaining missing values

After the above imputations, the only remaining missing column was `store_and_fwd_flag`. Since this is a binary categorical field (Y/N), missing values were filled with `"N"` (most trips are not store-and-forward; and this keeps the feature consistent).

Finally, a full missingness check confirmed **0 total missing values** across the dataset.

2.3. Handling Outliers and Standardising Values

2.3.1. Check outliers in payment type, trip distance and tip amount columns

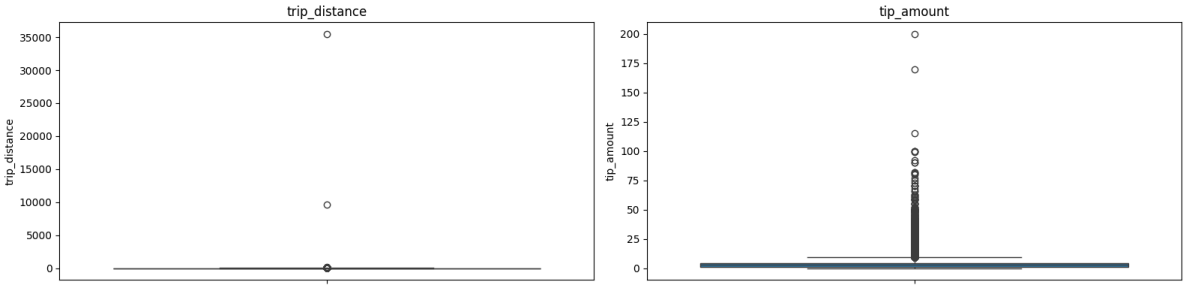
Outlier analysis (before fixing)

A percentile-based descriptive summary was computed (`1%, 5%, 95%, 99%`) to detect unusually large values. Key observations:

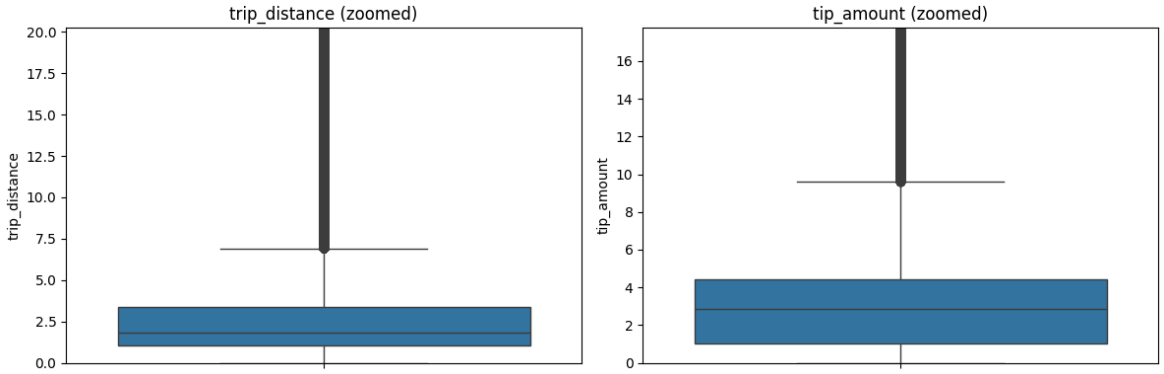
- trip_distance shows extreme values: while the 99th percentile is ~20 miles, the maximum distance is extremely large (tens of thousands), indicating clear erroneous entries.
- tip_amount is mostly small (median ~2–3, 99th percentile ~18), but the maximum reaches 200, which suggests rare but potentially extreme/outlier cases.
- payment_type includes values outside valid categories (notably 0, which is not defined in the dictionary and should be treated as invalid).

To support these observations, boxplots were drawn for:

- `trip_distance`
- `tip_amount`



A zoomed-in view (up to 99th percentile) was also plotted to better visualise the typical range and show how extreme points sit far outside the normal distribution.



Fixing the outliers (rule-based cleaning)

Based on the outlier analysis, rules were applied to remove records that are **highly likely to be data-entry or logging errors** rather than real trips.

Step 1: Passenger count filtering

Trips with very large passenger counts are rare and may represent errors. To keep the analysis realistic and consistent with standard taxi capacity, trips were limited to **passenger_count ≤ 6** (removing 7+ cases).

Step 2: Outlier masks applied

The following conditions were used to flag invalid/outlier rows:

1. **Near-zero distance but very high fare**
   - `trip_distance` nearly 0 but `fare_amount > 300`
     This violates basic fare logic and indicates wrong entry.

2. **Zero distance & zero fare but different pickup/dropoff zones**
   - If pickup and dropoff zones differ, both distance and fare being 0 is inconsistent.

3. **Extremely large trip distances**
   - `trip_distance > 250` miles is treated as unrealistic for typical NYC taxi trips.

4. **Invalid payment type**
   - `payment_type == 0` is not defined in the payment dictionary, so those rows were removed.

5. **Invalid trip time ordering**
   - `dropoff_datetime < pickup_datetime` indicates incorrect timestamps.

6. **Extreme tip amounts**
   - A **data-driven threshold** was used: values above the **99.9th percentile** of `tip_amount` were treated as extreme outliers to prevent them from skewing tip-related conclusions.

Outcome: The resulting dataframe (`df_sampled_300k_cleaned_and_handled`) retained realistic trips and removed clearly inconsistent entries that could distort later insights such as speed estimation, fare-per-mile comparisons, and tipping behaviour.

Standardising values (format consistency)

After cleaning, data types were reviewed to ensure each variable is in an appropriate format for analysis:

- Date/time columns remain in datetime format (`tpep_pickup_datetime`, `tpep_dropoff_datetime`)
- Monetary and continuous numeric fields are float (`fare_amount`, `trip_distance`, `tip_amount`, etc.)
- Categorical indicator fields remain as object/categorical (`store_and_fwd_flag`)

A quick uniqueness check for `store_and_fwd_flag` confirmed values are already standardised (`'N'`, `'Y'`), so no further transformation was required.

3. Exploratory Data Analysis

3.1. General EDA: Finding Patterns and Trends

3.1.1. Classify variables into categorical and numerical

Categorical (discrete labels / IDs):

- `VendorID` (vendor category)
- `RatecodeID` (rate category)
- `PULocationID` (pickup zone ID)
- `DOLocationID` (dropoff zone ID)
- `payment_type` (payment category)
- `store_and_fwd_flag` (Y/N flag)

Datetime / Temporal:

- `tpep_pickup_datetime`
- `tpep_dropoff_datetime`

Numerical (continuous or measurable quantities):

- `passenger_count`
- `trip_distance`
- `pickup_hour` (numeric hour derived from pickup time)
- `trip_duration` (pickup → dropoff duration)
- **Monetary variables (all numerical)**: `fare_amount`, `extra`, `mta_tax`, `tip_amount`, `tolls_amount`, `improvement_surcharge`, `total_amount`, `congestion_surcharge`, `airport_fee`

3.1.2. Analyse the distribution of taxi pickups by hours, days of the week, and months

	hour	pickups
0	0	8010
1	1	5384
2	2	3595
3	3	2321
4	4	1483
5	5	1556
6	6	3811
7	7	7746
8	8	10707
9	9	12389
10	10	13549
11	11	14847
12	12	15975
13	13	16347
14	14	17718
15	15	18304
16	16	18126
17	17	19417
18	18	20461
19	19	18367
20	20	16299
21	21	16249
22	22	14960
23	23	11800

	day	pickups
0	Monday	36200
1	Tuesday	42081
2	Wednesday	44366
3	Thursday	45339
4	Friday	43112
5	Saturday	42017
6	Sunday	36306

	month	pickups
0	1	23460
1	2	22048
2	3	26218
3	4	25139
4	5	26700
5	6	25179
6	7	21985
7	8	21356
8	9	20972
9	10	26195
10	11	25163
11	12	25006

Hourly trend (time of day)

Pickups are **lowest in the early morning hours** (roughly 2 AM - 5 AM), then begin rising sharply after **6 - 7 AM**. Demand stays high through the day and reaches its **peak in the evening**, with the **highest pickup volume around 6 PM (18:00)**. After 8 - 9 PM, pickups gradually decline but remain relatively active into late night compared to the early-morning low period.

Interpretation: This aligns with typical city travel patterns - morning commute increases demand, daytime trips remain steady, and evening commute + leisure travel drives the peak.

Day-of-week trend

Pickups are **highest on weekdays**, with the strongest demand seen from **Tuesday to Thursday**. **Friday** remains high, while demand dips on **Sunday** (lowest) and is moderate on **Saturday**.

Interpretation: Weekday work-related travel drives consistently high trip volumes, while Sunday demand drops due to reduced commuting and business activity.

Monthly trend

Pickups are **fairly consistent across months**, with noticeable upticks in a few months (the table shows higher values around months like **March, May, October**), and slightly lower activity in some others (like **February, July/September** in the output). Overall, the variation is **not extreme**, suggesting taxi demand is steady year-round with mild seasonal effects.

Interpretation: Small month-to-month differences may reflect seasonality (weather, holidays, tourism), but demand remains broadly stable across the year.

3.1.3. Filter out the zero/negative values in fares, distance and tips

	count	mean	std	min	25%	50%	75%	max	zeros	negatives
fare_amount	289421.0	19.626760	17.772745	0.0	9.30	13.50	21.90	1375.00	79	0
tip_amount	289421.0	3.541898	3.817801	0.0	1.00	2.86	4.45	30.00	64685	0
total_amount	289421.0	28.714935	22.252333	0.0	15.96	21.00	30.65	1435.19	41	0
trip_distance	289421.0	3.428573	4.481219	0.0	1.05	1.78	3.37	204.86	3553	0

From the summary of financial variables (`fare_amount`, `tip_amount`, `total_amount`, `trip_distance`):

- **Negative values**: None were observed in these selected columns (negatives = 0), indicating monetary fields are largely clean after earlier preprocessing.

- **Zero values**: Zeros exist in all four variables:
  - `fare_amount`: 79 zeros
  - `tip_amount`: 66,485 zeros (very common — many trips have no tip)
  - `total_amount`: 41 zeros
  - `trip_distance`: 3,553 zeros (possible for same-zone pickup/dropoff or potential logging issues)

What was filtered?

To analyse "paid trips" more realistically, a filtered dataset was created by **removing records where** `fare_amount`, `tip_amount`, **or** `total_amount` **are zero** (keeping only strictly positive values). This produces a cleaner subset for fare/tip behaviour analysis because:

- `fare_amount = 0` or `total_amount = 0` usually indicates **invalid/incorrect records** (or special cases not meaningful for revenue analysis).
- `tip_amount = 0` is valid behaviour, but excluding it is useful when the goal is to study tipping patterns only among trips where a tip occurred.

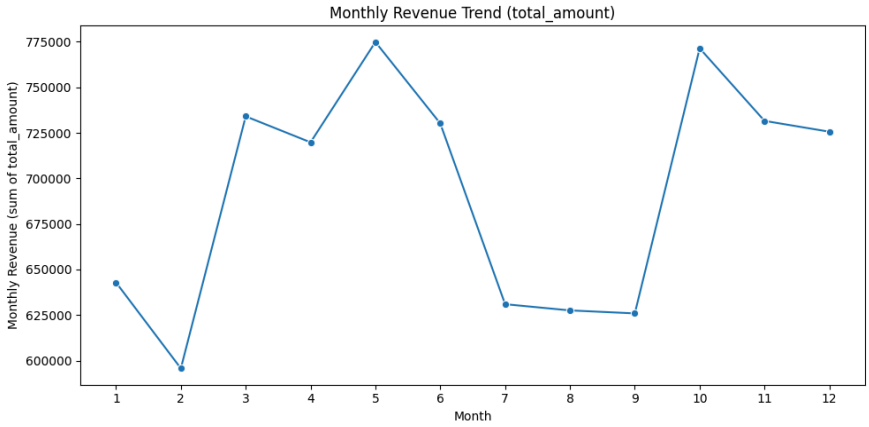
Note on `trip_distance = 0`: Zero distance can be legitimate (pickup and dropoff in the same zone), so it's better to **not blindly drop** all zero-distance trips unless the analysis specifically needs distance-based relationships (e.g., fare vs distance correlation). In those cases, dropping `trip_distance = 0` is reasonable to avoid division-by-zero or misleading fare-per-mile calculations.

Result: A “non-zero financial subset” was generated for cleaner financial/tipping analyses, while distance-based filtering can be applied only when required by the specific task.

3.1.4. Analyse the monthly revenue trends

Monthly revenue was computed as the sum of `total_amount` grouped by pickup month (using the filtered “non-zero financial” dataset).

	month	monthly_revenue
0	1	642905.26
1	2	595753.07
2	3	734039.73
3	4	719794.71
4	5	774829.09
5	6	730160.70
6	7	630973.92
7	8	627567.23
8	9	625927.27
9	10	771381.51
10	11	731561.40
11	12	725613.48



Observed pattern:

- Revenue is **moderate in Jan - Feb** (≈ 0.60–0.64M).
- It **rises sharply from March** and stays strong through **May - June** (peak around **May**, with March/June also high).
- There is a **dip in July - September** (lowest around **Aug/Sep**).
- Revenue **picks up again in Oct - Dec**, with **Oct/Nov/Dec** all relatively high.

Interpretation:
This suggests a **seasonal effect** in revenue:

- **Spring/early summer** shows higher total takings (more trips, longer trips, or higher fares).
- **Late summer** shows reduced revenue compared to peak months.
- **Year-end** demand strengthens again, which could be linked to holiday season activity and increased city movement.

3.1.5. Find the proportion of each quarter’s revenue in the yearly revenue

Quarter-wise revenue was calculated by:

1. assigning each trip to a **quarter** using pickup datetime,
2. summing `total_amount` per quarter, and
3. dividing each quarter’s total by the **yearly total** to get the revenue share.

	quarter	revenue_proportion
0	1	0.237374
1	2	0.267707
2	3	0.226757
3	4	0.268161

Interpretation:

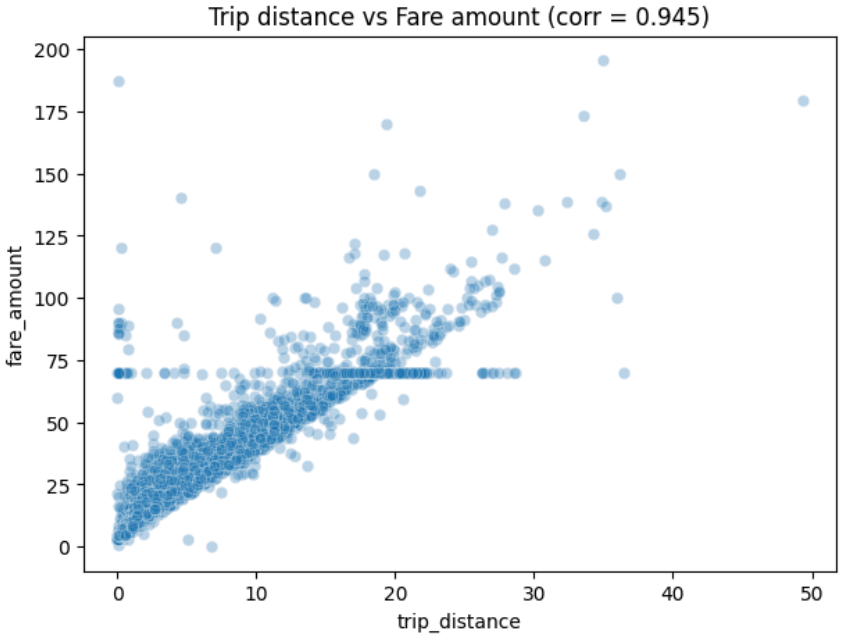
- The highest revenue contribution comes from **Q4 (~26.8%)** and **Q2 (~26.8%)**, indicating stronger revenue in Apr - Jun and Oct - Dec.
- **Q3 is the weakest (~22.7%)**, aligning with the dip you saw in **July - September** in the monthly trend.
- **Q1 is moderate (~23.7%)**, suggesting steady but not peak demand at the start of the year.

This shows a clear seasonal pattern where mid-year and year-end periods generate a larger share of annual taxi revenue.

3.1.6. Analyse and visualise the relationship between distance and fare amount

To study how fares change with distance, trips with `trip_distance = 0` were excluded (since they can distort distance-based relationships). The **Pearson correlation** between `trip_distance` and `fare_amount` was then computed.

Correlation result: r = 0.9456
This indicates a **very strong positive relationship** between distance and fare - as trip distance increases, fare amount generally increases.



What the scatter plot shows:

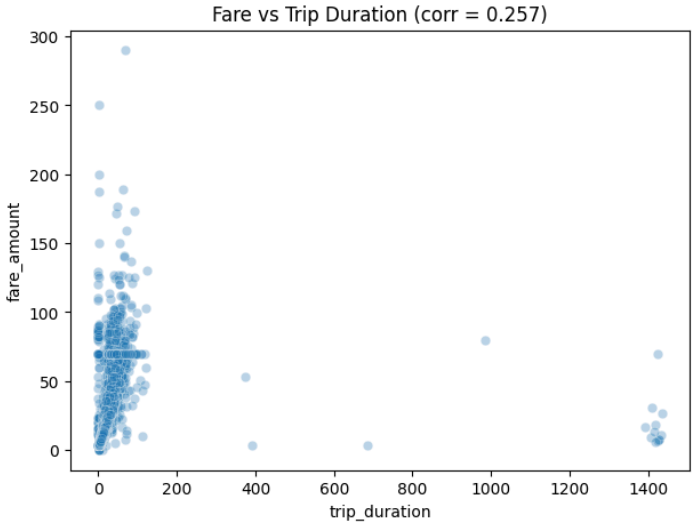
- A clear upward trend: longer trips tend to cost more.
- Some **spread** around the trend line, which is expected due to:
    - base fare + fixed surcharges,
    - traffic/time effects (not included in distance),
    - tolls/extras,
    - occasional unusual trips (outliers).

Business takeaway: Distance is one of the strongest drivers of fare in the dataset, making it a reliable variable for revenue estimation and pricing analysis (especially when combined with time-based factors).

3.1.7. Analyse the relationship between fare/tips and trips/passengers

1) `fare_amount` vs `trip_duration`

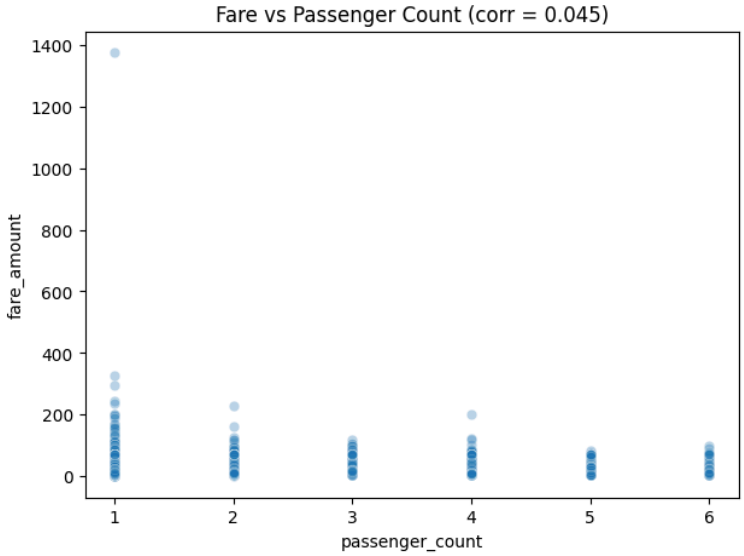
After computing `trip_duration` as the time difference between drop-off and pickup (in minutes) and filtering invalid/negative durations, the correlation between **fare** and **duration** is **weakly positive**.



This suggests that while longer trips *can* cost more, duration alone does not strongly determine fare because NYC taxi fare is influenced more by **distance** and fare rules, and duration can increase due to **traffic/congestion** without proportional increases in fare.

2) `fare_amount` vs `passenger_count`

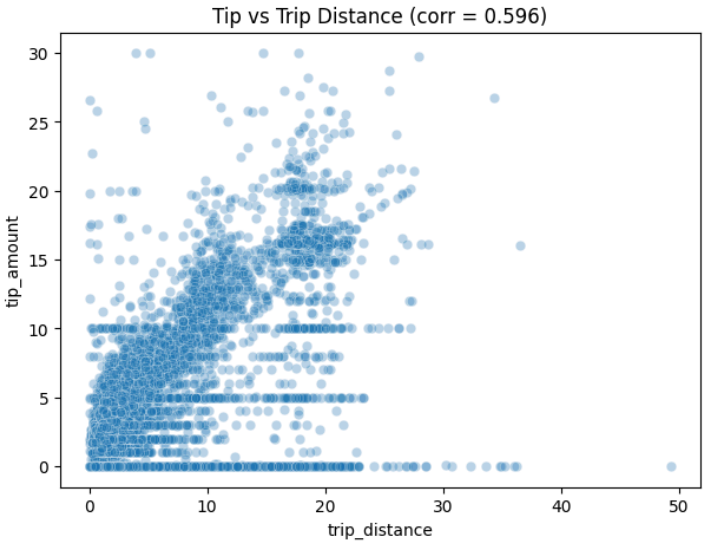
The correlation between **fare** and **passenger_count** is near zero.



This indicates passenger count does not materially affect the fare amount for NYC taxi trips. The scatter plot also supports this: for each passenger count (1 - 6), fares spread across similar ranges. This aligns with how taxi fares are generally based on trip characteristics (distance/time/surcharges), not number of passengers (within allowed capacity).

3) `tip_amount` **vs** `trip_distance`

The correlation between **tip amount** and **trip distance** is moderately positive.



This implies passengers tend to tip more on longer trips, likely because longer trips have higher fares, and tipping often scales with the perceived service value or fare total. The scatter plot shows a clear upward trend, although there is variability - tips are influenced by payment method, passenger preference, trip purpose, and time of day.

Overall insight:

- **Distance strongly drives fare**, but **duration is a weaker driver** due to traffic variability and fare structure.
- **Passenger count has negligible influence** on fare.
- **Tips increase with trip distance**, but remain behaviour-dependent and variable.

3.1.8. Analyse the distribution of different payment types

I analysed the distribution of taxi trips across different payment modes using the `payment_type` field (mapped to human-readable labels).

	payment_type	count	payment_label
0	1	235924	Credit card
1	2	49919	Cash
2	3	1441	No charge
3	4	2137	Dispute

From the sampled dataset, **credit card payments dominate overwhelmingly**, followed by **cash**. The remaining categories (**no charge** and **dispute**) form a **very small fraction** of total trips.

What this implies (business view):

- Since most trips are paid by **credit card, tip-related analysis is more meaningful on card transactions**, because tips are more consistently recorded digitally.
- **Cash trips** are still significant, but tips may be under-reported (not always captured in the dataset).

- **No charge** and **dispute** are rare and likely represent edge cases (promotional/adjustment trips or fare disputes), so they should not drive operational conclusions.

Overall insight: payment behaviour is highly skewed toward digital/card payments, which is useful for revenue, tipping, and customer behaviour analysis.

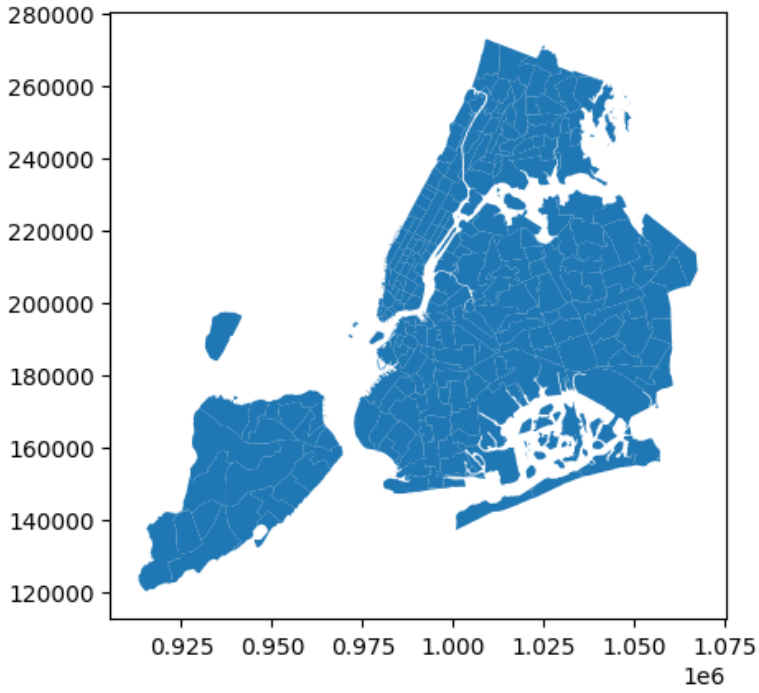
3.1.9. Load the taxi zones shapefile and display it

I loaded the NYC taxi zones shapefile using GeoPandas, which contains polygon boundaries for **263 taxi zones** across boroughs (Manhattan, Queens, Bronx, Brooklyn, Staten Island, plus EWR).

	OBJECTID	Shape_Leng	Shape_Area	zone	LocationID	borough	geometry
0	1	0.116357	0.000782	Newark Airport	1	EWR	POLYGON ((933100.918 192536.086, 933091.011 19...
1	2	0.433470	0.004866	Jamaica Bay	2	Queens	MULTIPOLYGON (((1033269.244 172126.008, 103343...
2	3	0.084341	0.000314	Allerton/Pelham Gardens	3	Bronx	POLYGON ((1026308.77 256767.698, 1026495.593 2...
3	4	0.043567	0.000112	Alphabet City	4	Manhattan	POLYGON ((992073.467 203714.076, 992068.667 20...
4	5	0.092146	0.000498	Arden Heights	5	Staten Island	POLYGON ((935843.31 144283.336, 936046.565 144...

Key fields used later:

- **LocationID**: unique zone identifier (this matches `PULocationID` and `DOLocationID` in trip records)
- **zone**: human-readable zone name
- **borough**: borough name
- **geometry**: polygon shape used to plot the map



The plotted map confirms the zones cover the full NYC area and provides the spatial layer needed for zone-level trip analysis and choropleth maps (e.g., trips per zone).

3.1.10. Merge the zone data with trips data

I **merged the taxi zones shapefile data** into the trip dataset by matching `PULocationID` (**trips**) with `LocationID` (**zones**) using a **left join**.

This adds geographic context to each trip by bringing in:

- **Pickup zone name** (`PU_zone`)
- **Pickup borough** (`PU_borough`)

A left join ensures **all trip records are retained**, even if a small number of `PULocationID` values don't find a matching zone (those would become missing in `PU_zone/PU_borough`).

After the merge, I can now perform **zone-level pickup analysis** (top pickup zones, hourly pickup trends by zone, choropleth maps, etc.) using meaningful zone/borough labels instead of just numeric IDs.

3.1.11. Find the number of trips for each zone/location ID

I **grouped the trip dataset** by `PULocationID` (pickup location ID) and **counted the number of records** in each group.

This gives the **total number of trips originating from each pickup zone/location**.

	PULocationID	trip_count
0	1	33
1	3	7
2	4	311
3	5	2
4	6	3
...	...	...
239	261	1498
240	262	3768
241	263	5660
242	264	2702
243	265	125

244 rows × 2 columns

The output is a 2-column table:

- `PULocationID`: numeric ID of the pickup zone
- `trip_count`: total trips that started from that zone in the sampled dataset

This summary is useful to:

- identify **high-demand pickup zones** (largest `trip_count`)
- later **merge back into the zones GeoDataFrame** to visualize demand using a **zone-wise choropleth map**.

3.1.12. Add the number of trips for each zone to the zones dataframe

I **merged the trip counts** (computed per `PULocationID`) **back into the zones GeoDataFrame** using:

- `zones.LocationID` (zone identifier in the shapefile)
- `trip_counts.PULocationID` (pickup zone ID in trip records)

	OBJECTID	Shape_Leng	Shape_Area	zone	LocationID	borough	geometry	trip_count
0	1	0.116357	0.000782	Newark Airport	1	EWR	POLYGON ((933100.918 192536.086, 933091.011 19...	33
1	2	0.433470	0.004866	Jamaica Bay	2	Queens	MULTIPOLYGON (((1033269.244 172126.008, 103343...	0
2	3	0.084341	0.000314	Allerton/Pelham Gardens	3	Bronx	POLYGON ((1026308.77 256767.698, 1026495.593 2...	7
3	4	0.043567	0.000112	Alphabet City	4	Manhattan	POLYGON ((992073.467 203714.076, 992068.667 20...	311
4	5	0.092146	0.000498	Arden Heights	5	Staten Island	POLYGON ((935843.31 144283.336, 936046.565 144...	2

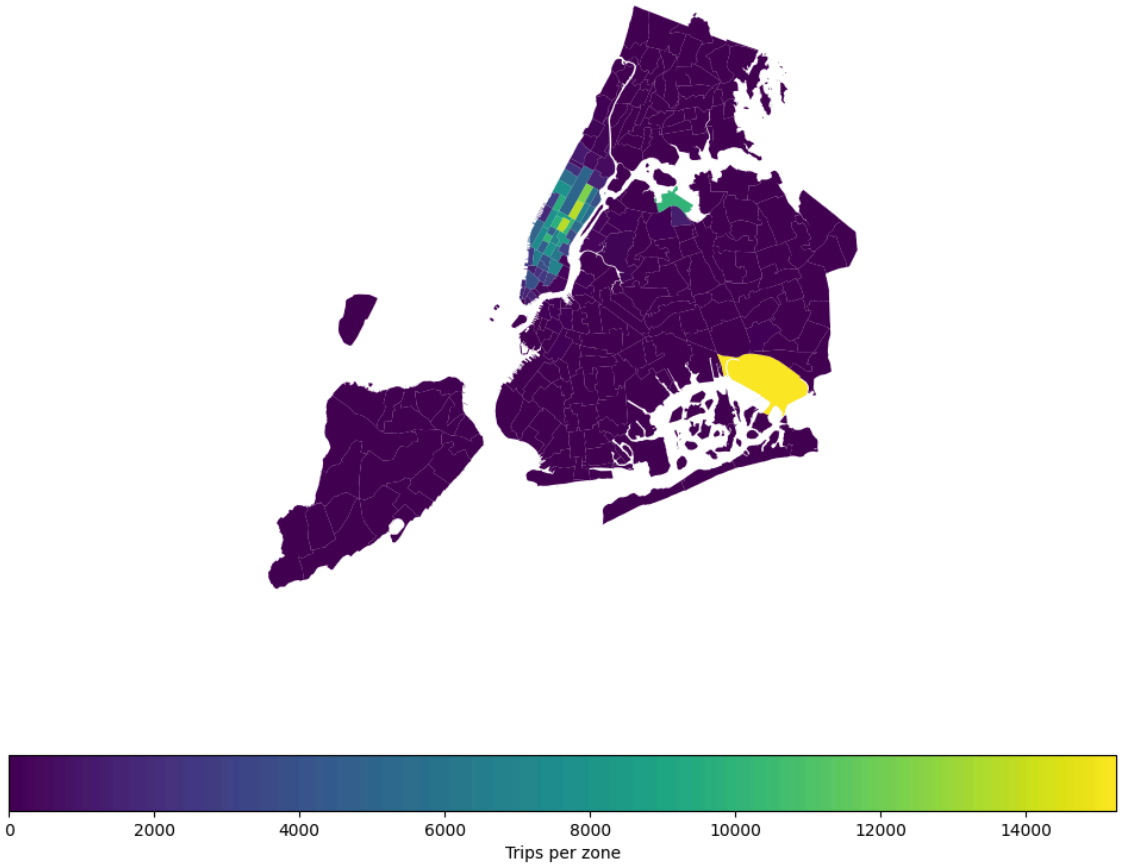
This adds a new column `trip_count` to each zone polygon, representing **how many trips originated from that zone** (in the sampled data).

I used a **left join** so that **all zones remain** in the GeoDataFrame, even if a zone has **no trips** in the sample.

Any missing counts after the merge were filled with **0**, meaning **no observed pickups** for that zone in the sampled dataset.

3.1.13. Plot a map of the zones showing number of trips

I plotted a choropleth map of NYC taxi zones where each zone is colored by the total number of trips originating from that zone (based on the pickup location). This provides a clear spatial view of demand concentration across the city**.**



From the map and the sorted zone table, trip activity is **highly concentrated in a small set of zones**, primarily in **Manhattan** (notably Midtown and surrounding areas), with **major airports such as JFK Airport and LaGuardia Airport** also showing very high trip volumes. In contrast, many outer zones (including several parts of Staten Island and lower-density areas) show relatively **low pickup volumes**.

This spatial distribution suggests that taxi demand is not uniform across NYC - certain commercial/transport hubs dominate pickups. These insights can later support:

- **supply positioning** (placing more cabs in high-demand zones),
- **dispatch planning** (anticipating peak loading areas),
- and **zone-based operational strategies** (e.g., airport/central business district focus).

`zones` DF sorted by the number of trips

LocationID	zone	borough	trip_count
132	JFK Airport	Queens	15250
237	Upper East Side South	Manhattan	13686
161	Midtown Center	Manhattan	13643
236	Upper East Side North	Manhattan	12277
162	Midtown East	Manhattan	10315
138	LaGuardia Airport	Queens	10186
186	Penn Station/Madison Sq West	Manhattan	10011
230	Times Sq/Theatre District	Manhattan	9611
142	Lincoln Square East	Manhattan	9541
170	Murray Hill	Manhattan	8534
163	Midtown North	Manhattan	8510
239	Upper West Side South	Manhattan	7906
48	Clinton East	Manhattan	7775
234	Union Sq	Manhattan	7719
68	East Chelsea	Manhattan	7532
164	Midtown South	Manhattan	6914
79	East Village	Manhattan	6817
141	Lenox Hill West	Manhattan	6765
249	West Village	Manhattan	6482
107	Gramercy	Manhattan	6132

3.1.14. Conclude with results

I completed the temporal, financial, and geographical analysis of the NYC taxi trip records to understand when demand is highest, where demand is concentrated, and which factors influence fares and tips.

- **Busiest hours (pickups by hour)**
  - Peak hour: **18 (6 PM)** with **20,461 pickups**
  - Lowest demand: **early morning (around 2 AM to 5 AM)**

- **Busiest days (pickups by day of week)**
  - Highest: **Thursday** with **45,339 pickups**
  - Lowest: **Monday** with **36,200 pickups** (Sunday is close at **36,306**)

- **Busiest months (pickups by month)**
  - Highest: **May (month 5)** with **26,700 pickups**
  - Next high: **October (month 10)** with **26,195 pickups**

- **Monthly revenue trend (total_amount)**
  - Highest monthly revenue: **May** with **774,829.09**
  - Next high: **October** with **771,381.51**
  - Mid-year months (July to September) are comparatively lower

- **Quarterly revenue share**
  - **Q4: 0.268161 (26.82%)** highest
  - **Q2: 0.267707 (26.77%)** very close to Q4
  - **Q1: 0.237374 (23.74%)**
  - **Q3: 0.226757 (22.68%)** lowest

- **How fare depends on distance, duration, passenger count**
  - **Trip distance vs fare**: very strong positive relationship, **corr = 0.9456**
  - **Trip duration vs fare**: weak positive relationship, **corr = 0.2572**
  - **Passenger count vs fare**: almost no relationship, **corr = 0.0454**

- **How tip amount depends on trip distance**
  - **Tip vs distance**: moderate positive relationship, **corr = 0.5957**

- **Busiest zones (highest trip counts)**
  - **JFK Airport (15,250)**
  - **Upper East Side South (13,686)**
  - **Midtown Center (13,643)**
  - **Upper East Side North (12,277)**
  - **Midtown East (10,315)**
  - **LaGuardia Airport (10,186)**
  - **Penn Station/Madison Sq West (10,011)**
  - **Times Sq/Theatre District (9,611)**
  - **Lincoln Square East (9,541)**
  - **Murray Hill (8,534)**

Overall, demand peaks in the evening commute hours, concentrates heavily in major Manhattan zones and airports, and fares are driven primarily by trip distance while tips increase moderately with distance.

3.2. Detailed EDA: Insights and Strategies

3.2.1. Identify slow routes by comparing average speeds on different routes

I calculated average route speed at each hour by grouping trips by **pickup_hour, pickup zone** (`PULocationID`) and **dropoff zone** (`DOLocationID`), then computing **avg_distance** and **avg_duration_min** for each route-hour combination.

Speed for a route at a given hour was computed as: **avg_distance** (miles) divided by average duration (hours), where average duration (hours) = **avg_duration_min** / 60.

	pickup_hour	PULocationID	DOLocationID	avg_distance	avg_duration_min	trips	avg_speed_mph
2656	0	263	68	4.640000	1434.666667	1	0.194052
2430	0	238	243	5.210000	1432.183333	1	0.218268
74	0	43	246	4.280000	723.941667	2	0.354725
1734	0	164	112	4.475000	720.550000	2	0.372632
1516	0	158	234	1.248750	181.245833	8	0.413389
...	...	...	...	...	...	...	...
60704	23	113	4	1.500000	723.191667	2	0.124448
62068	23	170	230	0.918750	182.991667	8	0.301243
62436	23	231	40	3.036667	476.638889	3	0.382260
63043	23	263	138	4.400000	683.866667	1	0.386040
60583	23	100	75	4.130000	486.316667	3	0.509545

120 rows × 7 columns

I then identified the slow routes by sorting, within each **pickup_hour**, the routes with the lowest **avg_speed_mph**. I also kept the **trips** count for each route-hour to avoid over-interpreting routes that appear only once or very few times.

Interpreting the output: routes with very low **avg_speed_mph** at certain hours likely represent congestion-heavy corridors at that time of day, bottlenecks near high activity zones (airports, Midtown, stations), or periods where traffic conditions consistently slow movement.

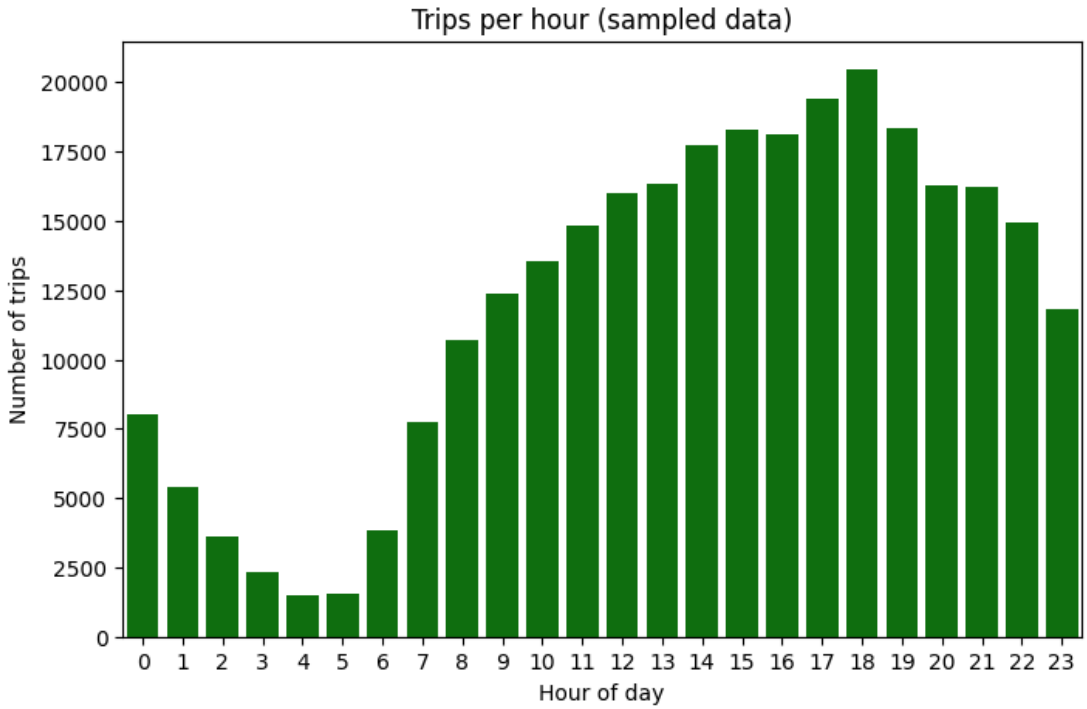
How identifying high-traffic, high-demand routes helps:

- Improves dispatching by positioning cabs closer to demand hotspots while avoiding known slow corridors during peak congestion
- Reduces passenger wait time by aligning supply with where pickups actually concentrate by hour
- Increases driver productivity by reducing time wasted in low-speed routes and improving trips per hour
- Helps adjust ETAs more accurately because delays become predictable by route and hour
- Supports pricing decisions since slow routes are time-expensive even if distance is short, which can justify time-based adjustments or peak-hour pricing

3.2.2. Calculate the hourly number of trips and identify the busy hours

I calculated hourly trip volume by extracting `pickup_hour` from `tpep_pickup_datetime` and counting trips per hour across the dataset.

The **busiest hour** is **18 (6 PM)** with **20,461 trips**, meaning demand peaks in the evening commute window.



The bar chart shows a clear daily pattern:

- **Lowest demand** happens during **late night to early morning** (roughly 2 to 5).
- Trips start rising after **morning hours** (6 onwards), stay high through the day, and peak around **evening** (17 to 19), then gradually decline at night.

This helps operationally because knowing the busiest `pickup_hour` supports:

- Better cab positioning before the peak hours
- Smarter staffing and dispatch planning
- Reduced passenger wait times during high-demand windows

3.2.3. Scale up the number of trips from above to find the actual number of trips

Why scaling was needed

- I worked on a sampled dataset where `sample_fraction = 0.05` (5% of trips).
- To estimate the **actual** number of trips, I scaled the sampled counts by `1 / sample_fraction`, so `estimated_actual_trips = trips / 0.05`.

Top 5 busiest hours and scaled estimates

	hour	trips	estimated_actual_trips
18	18	20461	409220
17	17	19417	388340
19	19	18367	367340
15	15	18304	366080
16	16	18126	362520

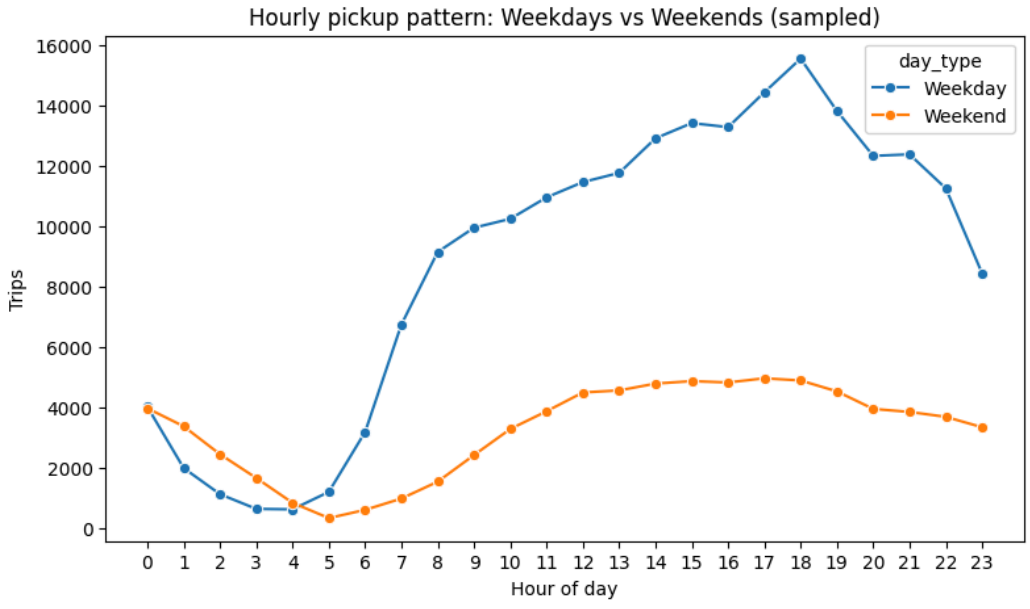
Key takeaway

- Demand was highest in the **late afternoon to evening** window, especially between `17` **and** `19`, with the peak at `18` (**6 PM**).

3.2.4. Compare hourly traffic on weekdays and weekends

I created `pickup_hour` from `tpep_pickup_datetime` and tagged each trip as `is_weekend` using `tpep_pickup_datetime.dt.dayofweek >= 5`.

I grouped trips by `pickup_hour` and `is_weekend` to compute hourly trip volumes (`trips`) and plotted the weekday vs weekend hourly pickup curves.



What I inferred from the pattern

- **Weekdays had consistently higher pickup volume than weekends** across most hours.
- **Weekday demand ramped up sharply after early morning**, stayed high through the day, and **peaked in the evening commute window** around 17 to 19.
- **Weekend demand was comparatively flatter**, with a **slower rise from morning**, a **moderate midday to evening bump**, and **no sharp commute-like spike**.
- **Late night to early morning hours** (roughly 1 to 5) were **quiet for both**, with weekends slightly higher than weekdays at some late-night hours.

How finding busy and quiet hours helps

- **Fleet positioning**: I could position more cabs in high-demand zones during busy windows (weekday evenings) and reduce idle supply during quiet windows.
- **Driver scheduling**: I could align shift start and end times with peaks, improving `trips_per_hour` and reducing downtime.
- **Better ETAs**: I could anticipate congestion-related slowdowns during peak hours and improve ETA accuracy by hour and day type.
- **Pricing strategy**: I could justify time-based or peak-hour adjustments when demand is high and supply is constrained.
- **Operational planning**: I could plan maintenance, breaks, and low-priority operations during consistently low-demand windows.

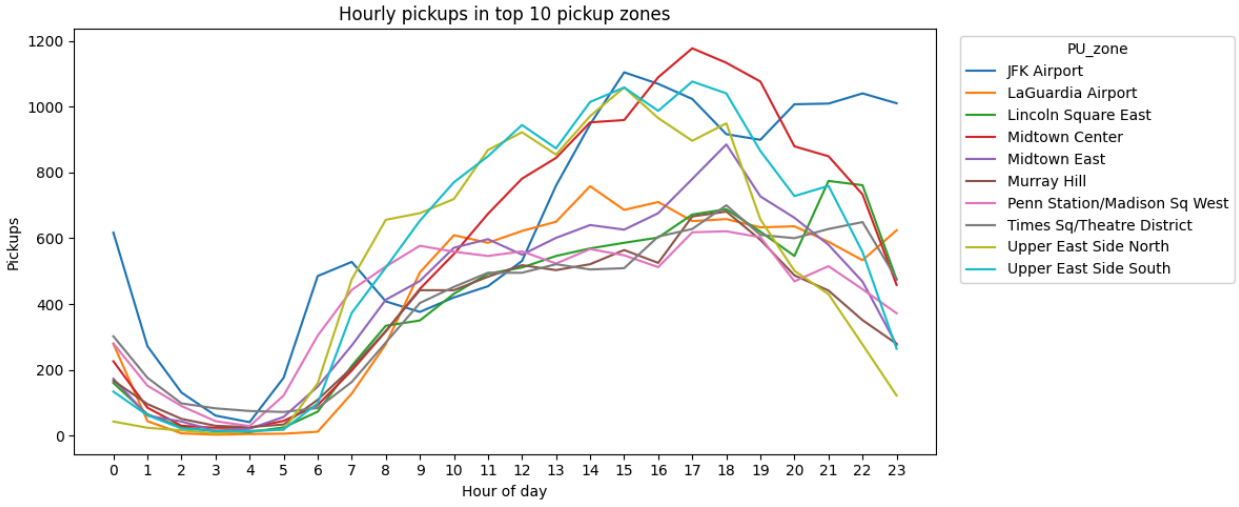
3.2.5. Identify the top 10 zones with high hourly pickups and drops

I identified the top 10 pickup zones by taking the highest overall counts of `PULocationID`, and I identified the top 10 dropoff zones by taking the highest overall counts of `DOLocationID`.

I then converted these IDs into human-readable zone names by using the merged zones data, so pickup zones were shown using `PU_zone` and dropoff zones were shown using `DO_zone` (with their corresponding `PU_borough` and `DO_borough`).

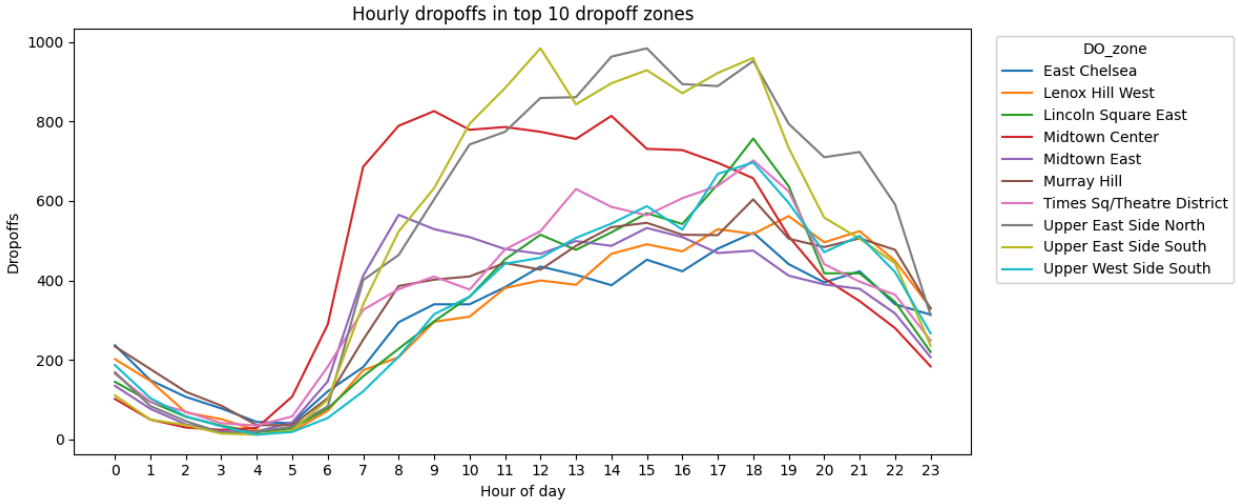
After selecting the top zones, I grouped the trips by `pickup_hour` and zone identifier to calculate:

- `pickups` per hour for each `PU_zone`
- `dropoffs` per hour for each `DO_zone`



Hourly pickup trend in top pickup zones

- Across the top pickup zones, pickups were **lowest during early morning hours** and increased steadily after morning.
- Most top pickup zones showed their strongest activity during **late afternoon and evening**, with higher concentration around `pickup_hour` in the **16 to 19** range.
- High-demand pickup hotspots included major transit and activity hubs such as **JFK Airport, LaGuardia Airport, Midtown Center, Times Sq/Theatre District,** and **Penn Station/Madison Sq West**, indicating that demand clustered around **airports, Midtown, and key transport hubs.**



Hourly dropoff trend in top dropoff zones

- Dropoffs also remained **low overnight** and increased through the day, with the strongest activity typically occurring from **midday to evening**.
- Popular dropoff areas included dense Manhattan zones such as **Midtown Center, Midtown East, Upper East Side North, Upper East Side South, and Times Sq/Theatre District**, showing that trip endpoints concentrated heavily in **commercial and residential Manhattan areas**.

Operational insight from pickup and dropoff patterns

Since both `PU_zone` and `DO_zone` demand peaked in the **evening hours**, cabs needed stronger availability in these zones before and during that window.

Airport zones (such as **JFK Airport** and **LaGuardia Airport**) consistently appeared as major pickup hotspots, so staging cabs near these zones during high-demand hours could reduce passenger wait time and improve driver utilization.

The overlap between high pickup and high dropoff zones suggested that many trips circulated within **Manhattan core areas**, which supported focused dispatch planning for Midtown and nearby zones.

3.2.6. Find the ratio of pickups and dropoffs in each zone

I calculated the **pickup to dropoff ratio** for each zone using:

```
pickup_drop_ratio = pickups / dropoffs
```

I first computed:

- Total **pickups per zone** using `PULocationID`
- Total **dropoffs per zone** using `DOLocationID`

I then merged both counts into a single table and computed `pickup_drop_ratio`.
Any cases where `dropoffs = 0` were handled safely to avoid division errors.

After mapping each `LocationID` to its **zone name**, I displayed:

- **Top 10 highest pickup/dropoff ratios**

	zone	LocationID	pickups	dropoffs	pickup_drop_ratio
0	East Elmhurst	70	1310.0	160	8.187500
1	JFK Airport	132	15283.0	3275	4.666565
2	LaGuardia Airport	138	10204.0	3533	2.888197
3	Penn Station/Madison Sq West	186	10013.0	6505	1.539277
4	Central Park	43	5025.0	3538	1.420294
5	Greenwich Village South	114	3804.0	2740	1.388321
6	West Village	249	6482.0	4760	1.361765
7	Midtown East	162	10315.0	8116	1.270946
8	Midtown Center	161	13644.0	11382	1.198735
9	Lincoln Square East	142	9541.0	8014	1.190542

These zones have **much higher pickups than dropoffs**, suggesting they act like **trip origin hubs**.
Examples from the result included **East Elmhurst**, **JFK Airport**, and **LaGuardia Airport**, which makes sense because airport areas generate many departures.

- **Top 10 lowest pickup/dropoff ratios**

	zone	LocationID	pickups	dropoffs	pickup_drop_ratio
0	Oakwood	176	0.0	1	0.0
1	Great Kills	109	0.0	4	0.0
2	New Dorp/Midland Beach	172	0.0	2	0.0
3	Mariners Harbor	156	0.0	6	0.0
4	Broad Channel	30	0.0	5	0.0
5	West Brighton	245	0.0	6	0.0
6	City Island	46	0.0	7	0.0
7	NaN	57	0.0	4	0.0
8	Stapleton	221	0.0	3	0.0
9	Van Cortlandt Park	240	0.0	6	0.0

Many of these had ratios equal to **0**, meaning **pickups were 0 while dropoffs existed** in the sampled data. These zones behave more like **destination-heavy zones** or zones with very low pickup activity in the sample.

Important data issue noted

- `LocationID = 57` showed **zone = NaN** because of a **typo in the source zones data**, where `LocationID = 56` **appears twice**, causing **57 to have no matching zone name** during mapping.

3.2.7. Identify the top zones with high traffic during night hours

I filtered trips to only **night hours** where `pickup_hour` was between **23 to 5** and then separately counted trips by `PULocationID` (pickups) and `DOLocationID` (dropoffs). I mapped each `LocationID` to its corresponding **zone name** and listed the **top 10 night pickup zones** and **top 10 night dropoff zones**.

Top night pickup zones (11 PM to 5 AM)

	zone	PULocationID	night_pickups
0	East Village	79	2497
1	JFK Airport	132	2308
2	West Village	249	1939
3	Clinton East	48	1704
4	Lower East Side	148	1571
5	Greenwich Village South	114	1364
6	Times Sq/Theatre District	230	1282
7	Penn Station/Madison Sq West	186	1089
8	LaGuardia Airport	138	968
9	Midtown South	164	963

Top night dropoff zones (11 PM to 5 AM)

	zone	DOLocationID	night_dropoffs
0	East Village	79	1345
1	Clinton East	48	1093
2	Murray Hill	170	1017
3	East Chelsea	68	970
4	Gramercy	107	880
5	Lenox Hill West	141	842
6	West Village	249	786
7	Times Sq/Theatre District	230	713
8	Yorkville West	263	713
9	Sutton Place/Turtle Bay North	229	695

Key night-time insight

- Night demand concentrated heavily in **Manhattan nightlife and residential zones** like **East Village, West Village, Clinton East, Murray Hill, Gramercy**, and also in **airport zones** like **JFK Airport** and **LaGuardia Airport**.
- This suggests night operations should focus on **dense Manhattan corridors** plus **airport connectivity** to reduce wait times and improve cab availability during late hours.

3.2.8. Find the revenue share for nighttime and daytime hours

I segmented trips into **night hours** `11PM–5AM` (`pickup_hour` >= `23` or <= `5`) and **daytime hours** `6AM-10PM` (`pickup_hour` between `6` and `22`). I then summed `total_amount` in each segment and computed each segment's **revenue share** as its revenue divided by the total revenue across both segments.

	time_period	revenue	revenue_share
0	night (11PM-5AM)	1001365.51	0.120491
1	day (6AM-10PM)	7309339.61	0.879509

- **Key takeaway**
  - The majority of revenue was generated during **daytime hours** `6AM–10PM`, while **nighttime** `11PM–5AM` contributed a much smaller share of total revenue.

3.2.9. For the different passenger counts, find the average fare per mile per passenger

I filtered trips to keep only valid records with `trip_distance > 0` and `passenger_count > 0`. For each trip, I computed `fare_per_mile = fare_amount / trip_distance`, then normalised it per passenger as `fare_per_mile_per_passenger = fare_per_mile / passenger_count`. Finally, I grouped by `passenger_count` and calculated the **average** `fare_per_mile_per_passenger`.

Results (avg fare per mile per passenger)

	passenger_count	avg_fare_per_mile_per_passenger
0	1.0	10.793718
1	2.0	5.947578
2	3.0	3.163639
3	4.0	3.391669
4	5.0	1.857500
5	6.0	1.357413

Key takeaway

- The **average fare per mile per passenger generally decreased** as `passenger_count` increased, meaning the cost per passenger-mile was highest for solo riders and lower for shared rides.

3.2.10. Find the average fare per mile by hours of the day and by days of the week

I filtered to valid trips where `trip_distance > 0`.

- I calculated `fare_per_mile = fare_amount / trip_distance`.
- I computed:
  - Hourly average by grouping on `pickup_hour`
  - Day-wise average by grouping on `day_name`

Hourly pattern (average `fare_per_mile`)

- The highest values appeared in the **late afternoon and evening**, with a peak around **hour 16**.
- Lower values appeared during **early morning** and later night hours.

Day-of-week pattern (average `fare_per_mile`)

- The highest average `fare_per_mile` occurred on **Friday** and **Sunday**.
- Mid-week values were moderately high, and **Monday** tended to be relatively lower compared to peak days.

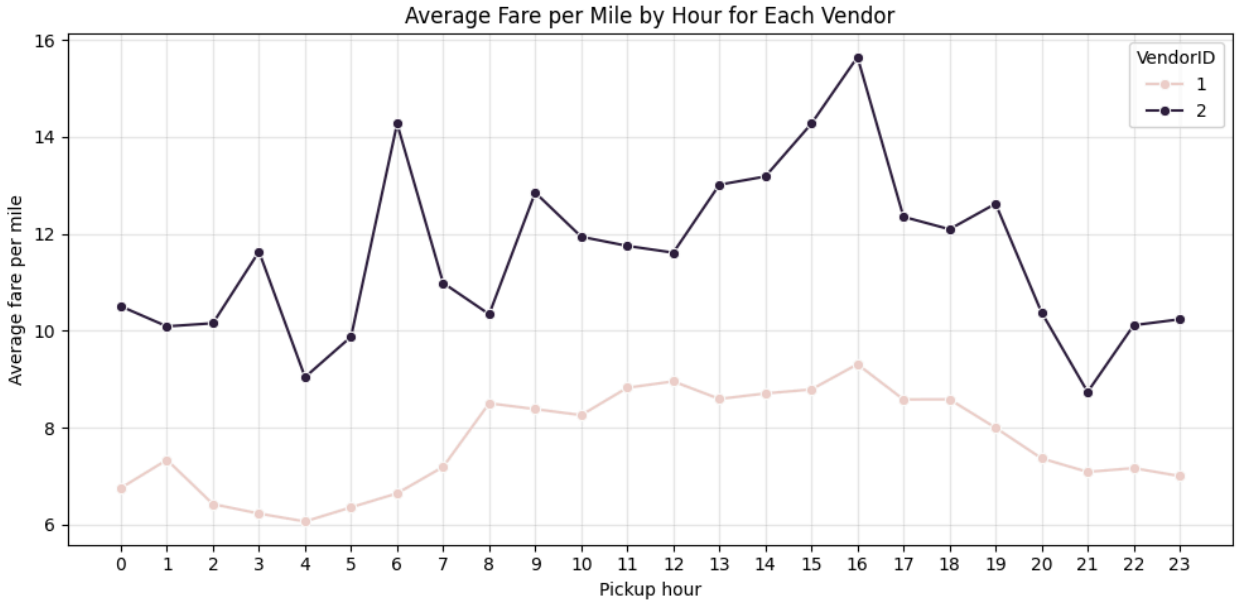
	pickup_hour	fare_per_mile
0	0	9.682258
1	1	9.458993
2	2	9.320798
3	3	10.377316
4	4	8.244206
5	5	8.803815
6	6	11.906846
7	7	9.849446
8	8	9.820113
9	9	11.594143
10	10	10.914806
11	11	10.946590
12	12	10.868622
13	13	11.769480
14	14	11.924283
15	15	12.769221
16	16	13.925342
17	17	11.347819
18	18	11.186504
19	19	11.477470
20	20	9.640597
21	21	8.355077
22	22	9.424008
23	23	9.504225

	day_name	fare_per_mile
0	Monday	10.249257
1	Tuesday	10.774885
2	Wednesday	11.315515
3	Thursday	10.471017
4	Friday	11.561379
5	Saturday	10.524791
6	Sunday	11.578242

3.2.11. Analyse the average fare per mile for the different vendors

AI filtered the dataset to trips with valid distance by keeping only rows where `trip_distance > 0`, since `fare_per_mile` would be undefined for zero-distance trips. I created a new metric `fare_per_mile` using `fare_amount / trip_distance` to standardise pricing across trips of different lengths.
I extracted the pickup time component `pickup_hour` from `tpep_pickup_datetime` to study hourly variation. I grouped trips by `VendorID` and `pickup_hour`, and computed the **mean** `fare_per_mile` for each vendor-hour combination to get `avg_fare_per_mile`.

I visualised this using a **line plot** with `pickup_hour` on the x-axis and `avg_fare_per_mile` on the y-axis, with separate lines for each `VendorID`, to compare how pricing per mile varied by vendor across the day.



From the plot, I observed that the vendors showed **different fare-per-mile patterns across hours**, indicating that `VendorID` influenced `fare_per_mile` depending on time of day, which is useful when comparing vendor-level pricing consistency and identifying hours where pricing diverged the most.

3.2.12. Compare the fare rates of different vendors in a distance-tiered fashion

I filtered the dataset to only valid trips by keeping rows where `trip_distance > 0`, since `fare_per_mile` cannot be computed for zero-distance trips.

I calculated `fare_per_mile` as `fare_amount / trip_distance` to compare vendor pricing in a distance-normalised way.

I created **distance tiers** using `trip_distance`:

- **0–2 miles**
- **2–5 miles**
- **>5 miles**

I grouped the data by `distance_tier` and `VendorID`, then computed the **mean** `fare_per_mile` for each combination to get `avg_fare_per_mile`.

The tier-wise results showed:

	distance_tier	VendorID	avg_fare_per_mile
0	0-2 miles	1	9.991237
1	0-2 miles	2	17.031865
2	2-5 miles	1	6.377112
3	2-5 miles	2	6.549023
4	>5 miles	1	4.400109
5	>5 miles	2	4.502742

Interpretation:

The largest vendor difference appeared in the **short-trip tier (0 - 2 miles)** where **Vendor 2 charged a much higher `fare_per_mile`** than **Vendor 1**.
For **medium (2 - 5 miles)** and **long trips (>5 miles)**, the vendors were **very close**, suggesting pricing differences were most pronounced on short rides.

3.2.13. Analyse the tip percentages

I calculated **tip percentage** as `tip_pct = (tip_amount / total_amount) * 100`, keeping only valid trips where `total_amount > 0` and `fare_amount > 0`.

Tip percentage by trip distance bucket

	distance_bucket	avg_tip_percentage
0	0-2	12.014161
1	2-5	12.290606
2	5-10	11.563488
3	10+	11.030475

This showed that tip percentage slightly decreased as `trip_distance` increased beyond mid range trips.

Tip percentage by passenger count

	passenger_count	avg_tip_percentage
0	1.0	12.075610
1	2.0	11.794944
2	3.0	11.229256
3	4.0	10.480406
4	5.0	11.874448
5	6.0	12.220562

Overall, tip percentage did not vary strongly with `passenger_count`, except a noticeable dip at `4`.

Tip percentage by pickup hour

Average tip percentage stayed fairly stable around **11% to 12.5%** across most `pickup_hour` values.

Lower tip percentages were observed during very early morning hours like `4` and `5` (around **10.2% and 10.05%**).

Slightly higher tip percentages appeared during late evening hours like `20` and `21` (around **12.49% and 12.51%**).

	pickup_hour	avg_tip_percentaget
0	0	11.865177
1	1	11.950898
2	2	11.718169
3	3	11.612025
4	4	10.209770
5	5	10.054366
6	6	11.102940
7	7	11.934015
8	8	12.167907
9	9	11.905032
10	10	11.710281
11	11	11.705553
12	12	11.801201
13	13	11.627486
14	14	11.700325
15	15	11.714823
16	16	11.736189
17	17	12.039743
18	18	12.285645
19	19	12.274977
20	20	12.487901
21	21	12.510463
22	22	12.396145
23	23	12.138662

What factors were linked to low tip percentages

	group	count	avg_distance	avg_fare	avg_total	avg_passengers	avg_pickup_hour
0	low_tip(<10%)	97611	3.556103	20.499183	26.533482	1.406911	14.036850
1	high_tip(>25%)	1481	1.761175	12.113707	25.939642	1.334909	13.492235

When comparing low tip trips (`tip_pct < 10%`) vs high tip trips (`tip_pct > 25%`)

Low tip group count was **97,611** trips vs high tip group count **1,481** trips

Low tip trips had higher averages for `trip_distance` (**3.56**) and `fare_amount` (**20.50**) than high tip trips (`trip_distance` **1.76**, `fare_amount` **12.11**)

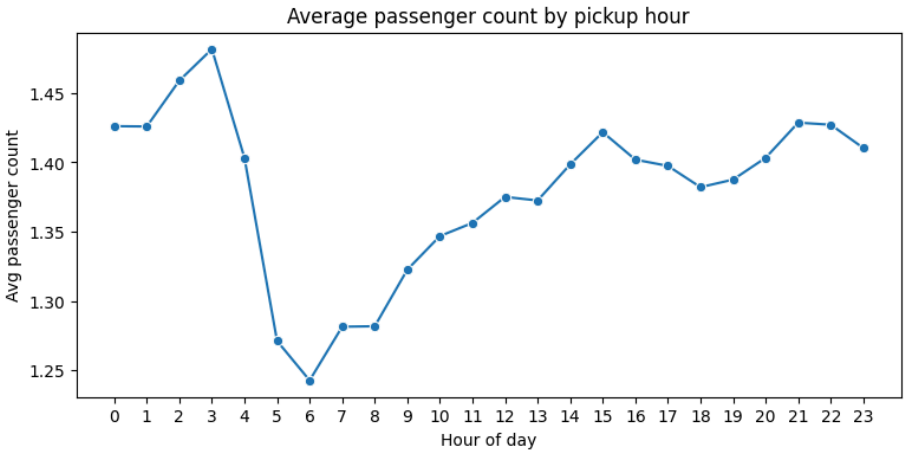
This suggested that lower tip percentages were more common in longer and higher fare trips, where riders may tip a smaller percentage even if the absolute tip is non zero.

3.2.14. Analyse the trends in passenger count

I analysed how `passenger_count` varied across `pickup_hour` and `day_name` by computing the average passenger count for each time slice.

- **Variation by hour (`pickup_hour`)**

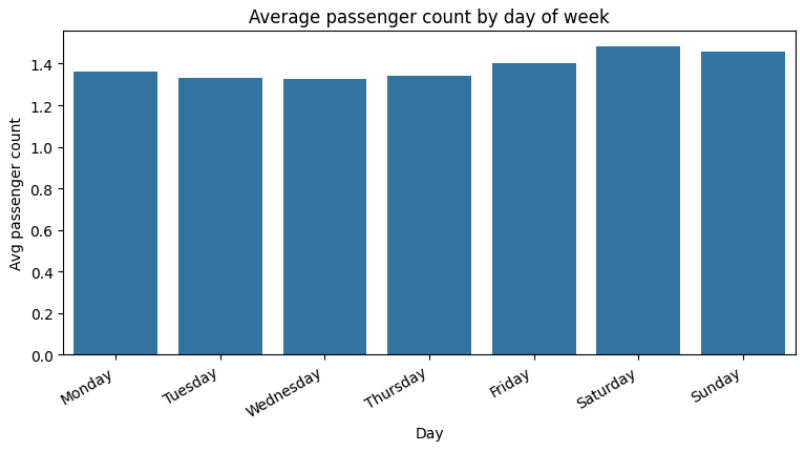
	pickup_hour	avg_passenger_count
0	0	1.426092
1	1	1.425892
2	2	1.458971
3	3	1.481689
4	4	1.403237
5	5	1.271208
6	6	1.242718
7	7	1.281436
8	8	1.281778
9	9	1.322625
10	10	1.346815
11	11	1.356301
12	12	1.375086
13	13	1.372484
14	14	1.398521
15	15	1.421766
16	16	1.401964
17	17	1.397435
18	18	1.382093
19	19	1.387543
20	20	1.403276
21	21	1.428642
22	22	1.427139
23	23	1.410339



- The **highest average `passenger_count`** occurred during **late night to early morning**, peaking around **3 AM** at **~1.48**.
- The **lowest average `passenger_count`** occurred in the **early morning commute window**, dipping around **6 AM** to **~1.24**.
- From **midday to evening**, the average passenger count stayed relatively stable, mostly within **~1.35 to ~1.43**, with a mild rise again in the **late evening**.

- **Variation by day of week (`day_name`)**

	day_name	avg_passenger_count
0	Monday	1.359475
1	Tuesday	1.331551
2	Wednesday	1.326038
3	Thursday	1.341648
4	Friday	1.400863
5	Saturday	1.483281
6	Sunday	1.459015



- **Weekend days had higher average `passenger_count`** compared to weekdays.
- **Highest: Saturday (~1.48)**, followed by **Sunday (~1.46)**.
- **Lowest: Tuesday and Wednesday (~1.33)**.
- Weekdays were tightly clustered around **~1.33 to ~1.40**, indicating relatively consistent single-rider dominance on working days.

- **Overall takeaway**
    - Passenger load was generally **close to 1 to 1.5 riders per trip**, suggesting most trips were **solo or paired rides**, with **weekends and late-night hours** showing slightly higher shared travel.

3.2.15. Analyse the variation of passenger counts across zones

I analysed how `passenger_count` varied across pickup zones by grouping trips by `PULocationID` (with zone labels `PU_zone` and `PU_borough`) and calculating the **mean** `passenger_count` per zone.

	PULocationID	PU_zone	PU_borough	avg_passenger_count
52	58	Country Club	Bronx	3.000000
53	59	Crotona Park	Bronx	2.000000
98	111	Green-Wood Cemetery	Brooklyn	2.000000
114	128	Inwood Hill Park	Manhattan	2.000000
178	195	Red Hook	Brooklyn	1.793103
10	12	Battery Park	Manhattan	1.789916
60	66	DUMBO/Vinegar Hill	Brooklyn	1.666667
4	6	Arrochar/Fort Wadsworth	Staten Island	1.666667
117	131	Jamaica Estates	Queens	1.625000
44	49	Clinton Hill	Brooklyn	1.600000
86	93	Flushing Meadows-Corona Park	Queens	1.576271
91	98	Fresh Meadows	Queens	1.571429
41	45	Chinatown	Manhattan	1.559682
234	256	Williamsburg (South Side)	Brooklyn	1.548387
177	194	Randalls Island	Manhattan	1.538462
239	261	World Trade Center	Manhattan	1.530040
213	232	Two Bridges/Seward Park	Manhattan	1.514950
233	255	Williamsburg (North Side)	Brooklyn	1.512500
191	209	Seaport	Manhattan	1.510714
39	43	Central Park	Manhattan	1.502886

The results showed that **average passenger count was usually close to 1 to 1.5** for most zones, meaning most trips were **single-passenger rides**.

A few zones had **noticeably higher** average passenger counts, indicating these areas more frequently had **group rides**. The highest average values observed were:

- **County Club (Bronx,** `PULocationID` **58): 3.0**
- **Crotona Park (Bronx,** `PULocationID` **59): 2.0**
- **Green-Wood Cemetery (Brooklyn,** `PULocationID` **111): 2.0**
- **Inwood Hill Park (Manhattan,** `PULocationID` **128): 2.0**

Other zones with relatively higher averages included places like **Red Hook (Brooklyn)**, **Battery Park (Manhattan)**, and **DUMBO/Vinegar Hill (Brooklyn)**, suggesting these areas may have more **shared or leisure-based trips** compared to typical commuter pickups.

To support mapping or geo-visualisation later, I merged the zone-level average passenger counts back into the zones GeoDataFrame so each `LocationID` carried its corresponding `avg_passenger_count`.

	LocationID	zone	borough	avg_passenger_count
0	1	Newark Airport	EWR	1.424242
1	2	Jamaica Bay	Queens	0.000000
2	3	Allerton/Pelham Gardens	Bronx	1.000000
3	4	Alphabet City	Manhattan	1.459807
4	5	Arden Heights	Staten Island	1.000000

3.2.16. Analyse the pickup/dropoff zones or times when extra charges are applied more frequently.

I analysed how often each surcharge was applied by checking the share of trips where each surcharge column was **greater than 0** for `extra`, `mta_tax`, `tolls_amount`, `improvement_surcharge`, `congestion_surcharge`, and `airport_fee`.

Overall surcharge frequency (applied count and % of trips):

	surcharge	applied_count	applied_percentage
3	improvement_surcharge	289314	99.963030
1	mta_tax	286886	99.124113
4	congestion_surcharge	267384	92.385832
0	extra	179354	61.969933
5	airport_fee	25417	8.782017
2	tolls_amount	23446	8.101002

I then checked **when** `extra` **was applied most frequently** by computing the percentage of trips with `extra > 0` for each `pickup_hour`.

`extra` **by time (hourly pattern)**:

	pickup_hour	percentage_extra_applied
0	0	95.742821
1	1	97.139673
2	2	96.968011
3	3	96.897889
4	4	90.424815

Finally, I checked **where `extra` was applied most frequently** by grouping by `PULocationID` and `PU_zone`, and calculating the percentage of trips with `extra > 0` per zone.

Top pickup zones where `extra` was most frequently applied:

	PULocationID	PU_zone	percentage_extra_applied
6	8	Astoria Park	100.000000
53	59	Crotona Park	100.000000
170	187	Port Richmond	100.000000
124	138	LaGuardia Airport	98.547025
134	148	Lower East Side	83.361515
64	70	East Elmhurst	82.645260
101	114	Greenwich Village South	79.642294
234	256	Williamsburg (South Side)	79.032258
73	79	East Village	77.306733
2	4	Alphabet City	76.527331
228	249	West Village	75.084850
49	54	Columbia Street	75.000000
173	190	Prospect Park	75.000000
233	255	Williamsburg (North Side)	73.750000
32	36	Bushwick North	73.333333

4. Conclusions

4.1. Final Insights and Recommendations

4.1.1. Recommendations to optimize routing and dispatching based on demand patterns and operational inefficiencies.

- I used route-hour level speeds, computed as `avg_speed_mph = avg_distance / (avg_duration_min / 60)`, to flag the slowest corridors by `pickup_hour`.
- I would avoid dispatching drivers into route-hour pairs with very low `avg_speed_mph` during peak periods, and instead reroute supply through nearby faster corridors to improve trips per hour.
- I also used `trips` per route-hour as a reliability check so decisions prioritize congestion patterns that occur frequently, not one-off anomalies.

4.1.2. Suggestions on strategically positioning cabs across different zones to make best use of insights uncovered by analysing trip trends across time, days and months.

- Demand peaked in the evening, with the busiest hour at `18` (6 PM) with **20,461 sampled pickups** (about **409,220** scaled using `sample_fraction = 0.05`), so I would stage the largest fleet buffer before `17` to `19`.

- I would keep strong coverage around high-demand zones like **JFK Airport (15,250)**, **Upper East Side South (13,686)**, **Midtown Center (13,643)**, and **LaGuardia Airport (10,186)** since they repeatedly appeared among the busiest pickup areas.

- For night hours `23` to `5`, I would shift supply toward zones with the highest night activity like **East Village (2,497 night pickups)** and **JFK Airport (2,308 night pickups)** to reduce wait times while maintaining utilization.

4.1.3. Propose data-driven adjustments to the pricing strategy to maximize revenue while maintaining competitive rates with other vendors.

- Since fare was strongly distance-driven (`corr(trip_distance, fare_amount) = 0.9456`), I would anchor pricing decisions primarily on `trip_distance` and time-of-day patterns, not on `passenger_count` because `corr = 0.0454`.

- Vendor pricing differed sharply on short trips, where `0-2 miles` had **Vendor 1: 9.99** vs **Vendor 2: 17.03** average `fare_per_mile`, so I would tighten short-trip pricing bands during peak hours to stay competitive.

- I would preserve margins on longer trips where vendors converged (`>5 miles` was about **4.40 to 4.50** `avg_fare_per_mile`), and focus on operational improvements (better ETAs, less congestion exposure) to support better tip behavior since `corr(tip_amount, trip_distance) = 0.5957`.